



UNIVERSIDAD
DE MÁLAGA

| uma.es

WatchGate

Sistema de *scoring* de riesgo para la revisión automatizada
de *pull requests* en pipelines CI/CD

Memoria del proyecto

Premios de la Cátedra de Ciberseguridad a la Innovación
en Ciberseguridad e Inteligencia Artificial

Universidad de Málaga · Cátedra de Ciberseguridad

Equipo solicitante: **Equipo WatchGate**

Javier Martín Jurado · Pablo Jiménez Castro · Pablo Ayllón García

Málaga, julio de 2026

Índice

Resumen ejecutivo	2
1. Problema	2
2. Solución propuesta	3
3. Uso previsto de técnicas de ciberseguridad e inteligencia artificial	5
4. Estado de desarrollo	5
5. Plan de trabajo (julio y agosto)	5
6. Hitos y entregables	6
7. Recursos necesarios	7
8. Resultado mínimo esperado	7
Anexo técnico	7
Referencias y enlaces técnicos	10

Resumen ejecutivo

Los ataques a la cadena de suministro de software a través de *pull requests* (PR) o cambios equivalentes (adopciones de paquetes, scripts de compilación) son una amenaza creciente y cada vez más sofisticada, agravada por el uso de IA generativa por parte de los propios atacantes para crear *payloads* adaptativos y difíciles de detectar. **WatchGate** propone un sistema de análisis y puntuación (*scoring*) de riesgo que se integra directamente en el pipeline de CI/CD (GitHub Actions, GitLab CI o un servidor Git interno) y evalúa cada PR combinando análisis estático, análisis de dependencias, reputación del autor y razonamiento semántico mediante modelos de lenguaje (LLM). El resultado es una puntuación explicada, capa por capa, que ayuda a mantenedores de proyectos *open source* y a equipos de desarrollo de empresa a decidir si un PR requiere revisión humana reforzada antes de fusionarse. El objetivo del periodo de desarrollo (julio y agosto de 2026) es entregar un prototipo funcional, ligero y fácilmente adoptable, con las cuatro capas de análisis operativas y validado contra un conjunto de casos de prueba reales y simulados.

1. Problema

Los repositorios de software, tanto proyectos *open source* como sistemas internos de empresa, dependen en gran medida de la revisión humana de cambios de código como principal barrera de control. Esta barrera tiene limitaciones estructurales: fatiga del revisor, sesgo de confianza hacia colaboradores habituales o hacia el historial aparente de un paquete, presión de tiempo y, sobre todo, la dificultad de detectar intención maliciosa cuando esta se disimula dentro de cambios que parecen legítimos.

Esta debilidad ya se explota de forma activa y a escala. El caso más reciente y revelador es la campaña «**Atomic Arch**», detectada en junio de 2026 contra el **Arch User Repository (AUR)**, el repositorio de paquetes mantenido por la comunidad de Arch Linux. Los atacantes explotaron el mecanismo legítimo de adopción de paquetes huérfanos para tomar el control de cientos de paquetes con reputación e historial ya establecidos y modificaron sus scripts de compilación (PKGBUILD) para desplegar de forma silenciosa un *infostealer* y, en sistemas con privilegios de *root*, un *rootkit*. En cuestión de días, los paquetes comprometidos pasaron de varios cientos a más de mil quinientos, en oleadas sucesivas con niveles crecientes de ofuscación.

Dos detalles de este caso justifican directamente la propuesta. Primero, algunos cambios maliciosos imitaban deliberadamente el nombre y el correo del autor de *commits* anteriores, generando una falsa sensación de continuidad: el ataque estaba diseñado para engañar una revisión basada en confianza aparente, no en el contenido real del cambio. Segundo, la propia comunidad de seguridad recurrió a modelos de IA para identificar variantes

que la revisión manual no había detectado: el investigador Nicolas Boichat empleó el modelo Gemma E2B ejecutado en local para localizar nuevas variantes ofuscadas del *payload*, confirmando que el razonamiento semántico automatizado ya es, en la práctica, parte de la respuesta a este tipo de amenazas.

El patrón no es exclusivo de un ecosistema. En 2024, el ataque a **XZ Utils** demostró que un actor malicioso puede ganarse la confianza de un proyecto durante meses antes de introducir una puerta trasera cuidadosamente disimulada. Y en 2026, la campaña **prrt-scan**, documentada por la firma de seguridad Wiz, mostró que un atacante puede abrir cientos de *pull requests* maliciosos contra repositorios de GitHub, generando mediante IA *payloads* que se adaptan al lenguaje y al *stack* tecnológico de cada objetivo. La respuesta defensiva no puede seguir dependiendo solo de revisión humana, de reglas estáticas fijas o de señales de reputación superficiales que, como demuestra Atomic Arch, pueden falsificarse deliberadamente: se requieren sistemas capaces de razonar sobre la intención semántica real del cambio, con la misma flexibilidad y velocidad que el atacante.

1.1 Estado del arte

Existen ya herramientas relevantes en este espacio, y es importante situar la propuesta frente a ellas:

- **Socket.dev** y **GuardDog** (Datadog, *open source*) analizan dependencias y paquetes en busca de indicadores de compromiso. Su foco es la *dependencia externa*, no el código introducido directamente en el PR.
- **BewAIre**, sistema interno de Datadog, utiliza LLM para detectar código malicioso en *pull requests* con resultados sólidos (95,5 % de detección en pruebas internas), pero es una herramienta cerrada y pensada para la infraestructura de una gran empresa.
- Herramientas comerciales como **Phylum**, **Aikido Security** o **Snyk** cubren el análisis de dependencias en modelos de suscripción orientados a empresas.

Herramienta	Abierto	Multiplataforma	Identidad vs. historial	Explicable por capas	Gobernanza LLM	Coste
Socket.dev / GuardDog	~ ¹	×	×	~ ²	n/a	Gratis / <i>freemium</i>
BewAIre (Datadog)	×	×	×	× ³	× ³	Interno, no disponible
Phylum / Aikido / Snyk	×	×	×	~	n/a	Suscripción
WatchGate	✓	✓ (A.0)	✓ (A.0.0, 3c)	✓ (A.1–A.3)	✓ (A.1, A.3.1)	Gratis, autoalojable

¹ GuardDog es abierto; Socket.dev, no · ² Solo cubre la capa de dependencias · ³ No documentado públicamente. Ninguna de estas soluciones ofrece, de forma abierta y ligera, razonamiento semántico sobre el propio *diff* del PR con una puntuación explicada, adoptable por proyectos pequeños o empresas sin presupuesto de seguridad dedicado. La diferenciación de WatchGate está en dos decisiones de diseño que ninguna herramienta anterior combina: la separación entre un núcleo agnóstico de plataforma y adaptadores finos (A.0), que hace que soportar GitHub, GitLab o un servidor Git interno sea añadir un adaptador y no reescribir el sistema; y una capa de reputación que cruza la identidad declarada del autor con su historial real de cuenta (A.0.0, 3c), dirigida específicamente contra el vector documentado en Atomic Arch. A ello se suma un principio de gobernanza poco habitual: el orquestador es determinista y la única agencia del sistema queda acotada dentro de la capa semántica (A.1, A.3.1); es decir, WatchGate no solo usa IA, sino que documenta cómo la mantiene auditable.

2. Solución propuesta

WatchGate es un sistema de *scoring* de riesgo multicapa que se ejecuta automáticamente al abrirse o actualizarse un *pull request*, integrado como una acción de CI/CD. En lugar de un veredicto binario (malicioso/benigno), frágil y poco fiable, combina varias señales independientes en una puntuación ponderada de 0 a 100 con un desglose explicado por capa:

- **Estática**: patrones de código peligrosos conocidos (*eval*, *exec*, ofuscación, escalada de privilegios), mediante reglas Semgrep/YARA.

- **Dependencias:** paquetes nuevos o modificados (*typosquatting*, scripts de instalación sospechosos, CVE conocidas), contra bases públicas (OSV, avisos de npm/PyPI).
- **Reputación del autor:** antigüedad de la cuenta, historial de contribuciones y consistencia entre la identidad declarada (nombre, correo) y el historial real de la cuenta, vía API de GitHub/GitLab.
- **Semántica (LLM):** razona sobre la intención del cambio en el contexto del proyecto, con independencia de la reputación aparente del autor, mediante *prompting* estructurado con salida JSON (nivel de riesgo, categoría y justificación), apoyado en un componente RAG sobre un corpus local de patrones de ataque conocidos (A.3.2).
- **Comportamental** (*extensión futura*): ejecución del cambio en un *sandbox* aislado con monitorización de llamadas de red y al sistema.

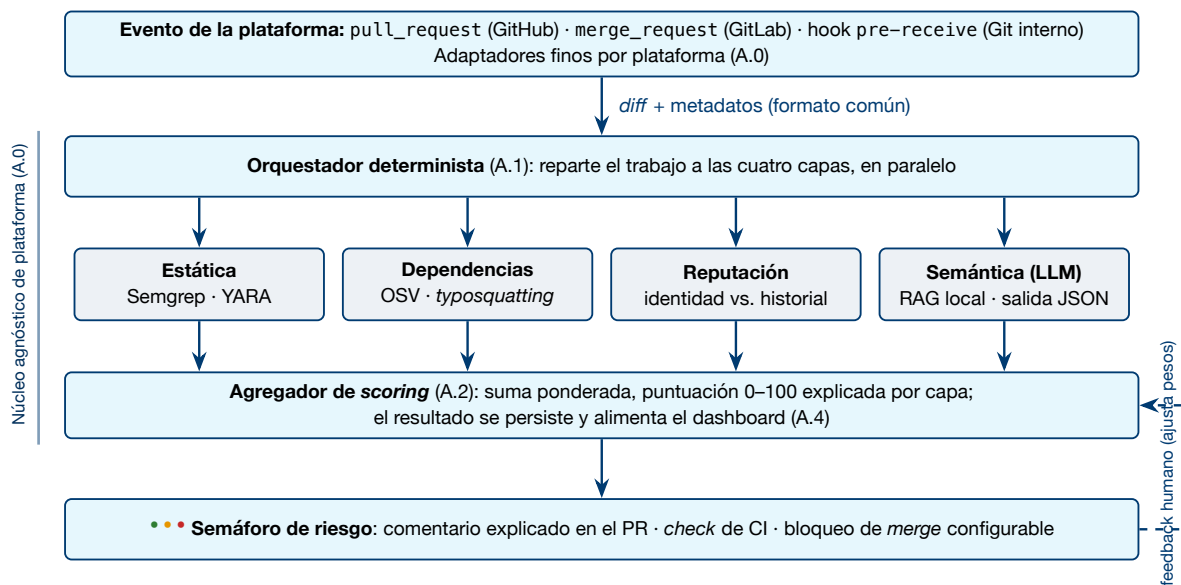


Figura 1. Arquitectura y flujo de ejecución de WatchGate: adaptadores finos por plataforma, núcleo agnóstico con orquestador determinista, cuatro capas de análisis en paralelo y agregador con salida explicada.

El caso Atomic Arch justifica por qué ninguna capa puede funcionar aislada: los atacantes falsificaron la identidad visible de los *commits* precisamente para superar una revisión basada en reputación superficial. Por eso la capa de reputación no se limita a leer nombre y correo, sino que los cruza con el comportamiento histórico real de la cuenta; y la capa semántica evalúa el contenido del cambio con independencia de quién parezca haberlo firmado. La seguridad del sistema está en la combinación ponderada de todas las señales.

El resultado se traduce en un semáforo de riesgo (verde/amarillo/rojo) publicado como comentario automático en el PR y como *check* de CI, pudiendo bloquear el *merge* en casos de riesgo alto. La explicabilidad por capas es un principio de diseño central: ningún equipo va a bloquear un despliegue por una cifra sin justificación. Ejemplo simplificado del comentario publicado:

[AMARILLO] WatchGate: Riesgo medio (47/100)

Estática	20/100	(peso 0.25)
Dependencias	10/100	(peso 0.20)
Reputación	40/100	(peso 0.15)
Semántica (LLM)	85/100	(peso 0.40)

Justificación (capa semántica):

"El cambio añade una llamada de red a un dominio externo no declarado en la documentación del proyecto, activada durante el proceso de build."

-> Se recomienda revisión humana reforzada antes de fusionar.

El sistema es además **modular**: cada capa es un componente independiente con la misma interfaz de entrada (*diff* + metadatos normalizados) y salida (puntuación 0–100 + justificación). Esto permite activar o desactivar

capas según el proyecto, incorporar capas futuras (como la comportamental) sin modificar el resto, y sustituir el proveedor de LLM sin afectar al agregador (A.0.0). Esta modularidad es la que hace viable entregar las cuatro capas operativas al final del periodo de desarrollo (secciones 4 y 8). La Figura 1 resume la arquitectura completa y el flujo de ejecución.

2.1 Dashboard de postura de seguridad agregada

Más allá del comentario por PR, el resultado mínimo garantizado (sección 8) incluye un *dashboard* web que agrega el historial de puntuaciones a lo largo del tiempo y entre repositorios, algo que GitHub/GitLab no ofrecen, al ser su vista por repositorio y por *check* individual. Cubre tres funciones: la vista histórica de puntuaciones con su desglose por capa; la interfaz del ciclo de *feedback* humano (paso 6 de A.1), donde un revisor marca un veredicto como correcto o falso positivo; y **autenticación SSO/OAuth con roles granulares por proyecto** (administrador de organización, mantenedor, revisor de solo lectura), de forma que una empresa pueda dar visibilidad a distintos equipos sin exponer el detalle de todos sus repositorios. Su alcance, arquitectura y priorización interna se detallan en el Anexo A.4.

3. Uso previsto de técnicas de ciberseguridad e inteligencia artificial

Ciberseguridad:

- Análisis estático mediante reglas Semgrep orientadas a patrones de ataque conocidos (exfiltración de datos, ejecución dinámica de código, escalada de privilegios).
- Detección de *typosquatting* y comprobación de dependencias contra bases públicas de vulnerabilidades (OSV, GitHub Advisory Database).
- Modelo de reputación que cruza la identidad declarada del autor con el comportamiento histórico real de la cuenta, para detectar discrepancias como las de Atomic Arch (identidad de *commits* imitada para simular continuidad).

Inteligencia artificial:

- LLM con *prompting* estructurado que analiza el *diff* en su contexto (tipo de proyecto, lenguaje, historial reciente) y genera una evaluación de intención con salida JSON: nivel de riesgo, categoría y justificación en lenguaje natural.
- Componente **RAG** sobre un corpus local de patrones de ataque documentados (XZ Utils, prt-scan, Atomic Arch, entradas relevantes de MITRE ATT&CK/CAPEC), que recupera los casos más similares al *diff* analizado como contexto adicional del LLM (A.3.2).
- Conjunto de ejemplos *few-shot* construido a partir de casos reales maliciosos y de cambios benignos, para calibrar el criterio del modelo.
- Sistema de ponderación combinada que integra las señales de todas las capas en una puntuación final, ajustable por proyecto.

4. Estado de desarrollo

El proyecto se encuentra en fase de diseño de arquitectura, detallado en el Anexo técnico: separación entre núcleo agnóstico de plataforma y adaptadores finos (A.0), interfaz común entre capas que garantiza la modularidad (A.0.0), adaptador de GitHub Action de referencia (A.0.1), flujo de ejecución con orquestador determinista (A.1), fórmula de *scoring* ponderado con umbrales configurables (A.2) y formato de salida estructurada de la capa semántica, incluyendo el uso acotado de herramientas por el LLM y el diseño del RAG local (A.3–A.3.2). No se ha iniciado aún la implementación de código; el plan de trabajo (sección 5) arranca la construcción del núcleo y las capas en la primera quincena de julio.

5. Plan de trabajo (julio y agosto)

Equipo de tres personas en tres líneas de trabajo paralelas (distribuibles según disponibilidad, no asignadas de forma fija) que siguen la arquitectura modular del Anexo técnico. Las tareas críticas (validación, demo) se adelantan a la penúltima semana de agosto, dejando la última solo para pulido y entrega. El resultado mínimo (sección 8) requiere las cuatro capas, el orquestador, el agregador, el RAG (A.3.2) y el control de coste básico (A.3.3), la herramienta de línea de comandos agnóstica de plataforma (A.0) y el *dashboard* completo (2.1, A.4). El único objetivo ampliado fuera del compromiso mínimo es el

cortocircuito de doble extremo (A.3.4), una optimización de coste no imprescindible.

- **Quincena 1 (primera mitad de julio):** Núcleo agnóstico de plataforma e interfaz común entre capas (A.0, A.0.0), formato de integración en CI/CD y módulo de análisis estático. Líneas: (1) núcleo, interfaz común y orquestador determinista (A.1) · (2) *prompts* y esquema JSON de la capa semántica (A.3); arranque de la capa de reputación (3c) · (3) reglas Semgrep/YARA y casos de prueba iniciales.
- **Quincena 2 (segunda mitad de julio):** Módulo semántico completo y capa de dependencias. Líneas: (1) agregador de *scoring* (A.2) y adaptador de GitHub Action (A.0.1) · (2) *prompting*, herramientas acotadas (A.3.1), *dataset few-shot* y corpus RAG con su indexado por *embeddings* (A.3.2) · (3) capa de dependencias (OSV, *typosquatting*), compaginada al ~50 % con el arranque del dashboard: modelo de datos histórico, GitHub OAuth y rol de revisor.
- **Del 1 al 10 de agosto:** Integración de las cuatro capas a través del orquestador. Líneas: (1) integración *end-to-end* núcleo + adaptador + agregador; control de coste básico (A.3.3) · (2) cierre de la capa de reputación y calibración de la semántica contra casos reales · (3) dedicación completa al dashboard: rol de mantenedor y proveedor OIDC genérico (A.4).
- **Del 11 al 22 de agosto:** Validación contra el conjunto de casos de prueba, corrección de errores, grabación de la demo y documentación técnica. Cierre funcional del dashboard (rol de administrador de organización, completando los tres roles de A.4). *Objetivo ampliado, solo si el avance lo permite:* cortocircuito de doble extremo (A.3.4).
- **Del 23 al 31 de agosto:** Pulido final, memoria final del periodo y preparación de la entrega.

5.1 Equipo solicitante y complementariedad

El **equipo WatchGate**, que comparte nombre con el proyecto, lo forman tres estudiantes del Grado en Ciberseguridad e Inteligencia Artificial de la Universidad de Málaga, con el tercer curso finalizado (cuarto a partir de septiembre de 2026), disponibilidad completa durante julio y agosto, y perfiles complementarios alineados con las tres líneas de trabajo del plan (cada integrante actúa como referente principal de una línea, sin perjuicio de la distribución flexible descrita arriba):

- **Javier Martín Jurado.** Un año y cuatro meses de experiencia en el grupo de investigación Mobilenet de la UMA, en proyectos de IA y ciberseguridad. Ganador del hackathon organizado por la ETSII (UMA) y Mercedes-Benz. Formador (*trainer*) en el Ciberbootcamp organizado por Google.org y la UMA. Inglés B2 (Cambridge). Referente de la línea 2 (capa semántica y de reputación).
- **Pablo Jiménez Castro.** Experiencia en detección de malware mediante el análisis de llamadas al sistema (*syscalls*) con redes neuronales LSTM. Ganador del hackathon organizado por la ETSII (UMA) y Mercedes-Benz. Certificados: *Machine Learning Operations (MLOps) for Generative AI* (Google) y *GenAI Powered Data Analytics Job Simulation* (Tata). Inglés C1 (Trinity College London). Referente de la línea 1 (núcleo, orquestador y agregador).
- **Pablo Ayllón García.** Experiencia en modelos de visión por computador para segmentación de lesiones en imágenes de resonancia magnética (MRI) y en redes LSTM para detección de crisis epilépticas en señales EEG. Líder del equipo de competiciones CTF «Hacker Factory», con experiencia en análisis de vulnerabilidades y seguridad ofensiva y defensiva. Conocimientos de administración de sistemas Linux, Windows y *firmware*. Certificados: Google Activate (Introducción al Desarrollo Web I y II, Desarrollo de Apps Móviles). Inglés B2 (Trinity College London). Referente de la línea 3 (capas estática/dependencias y dashboard).

El detalle completo de la trayectoria de cada integrante se acredita en los currículums breves que acompañan a la solicitud.

6. Hitos y entregables

- **Hito 1** (fin de la primera quincena de julio): núcleo agnóstico de plataforma (A.0) funcional como herramienta de línea de comandos sobre primitivas de git puro, con la capa estática operativa.
- **Hito 2** (fin de julio): módulo semántico integrado, con corpus RAG indexado (A.3.2), generando una evaluación de riesgo explicada por PR; arranque del dashboard (modelo de datos y GitHub OAuth con rol de revisor).
- **Hito 3** (10 de agosto): *scoring* combinado con las cuatro capas operativas y control de coste básico (A.3.3); dashboard con proveedor OIDC genérico y rol de mantenedor (A.4).
- **Hito 4** (22 de agosto): validación completa sobre el conjunto de casos de prueba, incluido el dashboard con

sus tres roles, y demo grabada, cerrada antes de la última semana para no depender de la disponibilidad simultánea de los tres integrantes.

- **Objetivo ampliado** (22 de agosto, no bloqueante): cortocircuito de doble extremo (A.3.4), única pieza del diseño fuera del compromiso mínimo.
- **Entregable final** (31 de agosto): repositorio *open source* documentado (dashboard incluido), vídeo demostrativo del flujo completo sobre un PR real e informe de validación sobre al menos 10 casos de prueba (benignos y maliciosos, simulados o documentados).

7. Recursos necesarios

- Acceso a API de modelo de lenguaje (créditos de inferencia) para desarrollo y prueba del módulo semántico, con control de coste por diseño (A.3.3).
- Entorno de pruebas / *sandbox* para validar *payloads* maliciosos simulados de forma segura y aislada.
- Acceso, si es posible a través de GSEC Málaga o de organizaciones colaboradoras, a un conjunto anonimizado de PR reales (benignos y de riesgo) para calibrar el sistema en condiciones realistas.
- Mentorización técnica en seguridad de la cadena de suministro de software, si está disponible.
- Para el dashboard: proveedor de identidad OAuth/SSO (GitHub OAuth o Auth0/Keycloak) y *hosting* ligero para el backend y la base de datos histórica.

8. Resultado mínimo esperado

Como resultado mínimo garantizado, el proyecto entregará el **núcleo de WatchGate como herramienta de línea de comandos, agnóstica de plataforma y operando sobre primitivas de git puro** (A.0), quedando el adaptador de GitHub Action como integración de referencia y no como parte del compromiso, con las **cuatro capas de análisis operativas** (estática, dependencias, reputación y semántica, esta última con su corpus RAG y control de coste básico), capaz de generar una puntuación de riesgo explicada sobre el *diff* de cualquier *pull request* de un repositorio de prueba, validada sobre al menos 10 casos representativos. El compromiso incluye además el **dashboard de postura de seguridad completo** (2.1, A.4): historial de puntuaciones, ciclo de *feedback* humano y autenticación SSO/OAuth con los tres roles granulares. El único objetivo ampliado fuera de este compromiso es el cortocircuito de doble extremo (A.3.4), una optimización de coste cuyo eventual retraso no compromete el resto del entregable.

Anexo técnico

A.0 Arquitectura: núcleo desacoplado y adaptadores por plataforma

Para que el sistema sea igual de viable en un repositorio *open source* sobre GitHub que en un control de versiones interno de empresa, la arquitectura se separa en dos capas (ver Figura 1, sección 2). El **núcleo** (agnóstico de plataforma) recibe únicamente primitivas de git puro (un *diff*, la lista de archivos modificados y metadatos de *commit* obtenibles con `git log/git show`) y ejecuta las capas de análisis y el agregador; no depende de ningún token ni API de plataforma. Los **adaptadores** (finos, uno por plataforma) traducen el evento nativo (`pull_request` de GitHub, `merge_request` de GitLab, *hook pre-receive* de un Git interno) a un formato común, invocan al núcleo y publican el resultado en el formato propio de la plataforma.

La capa de reputación es la única que necesita metadatos específicos de plataforma (historial de cuenta vía API); los recibe ya normalizados desde el adaptador, sin consultarlos directamente.

Esta separación permite desarrollar y validar el núcleo completo en local durante julio, sin depender de configuración de CI ni de tokens, dejando el adaptador como pieza final más mecánica; y hace que soportar GitLab o un Git interno sea añadir un adaptador nuevo, no reescribir la lógica de análisis.

A.0.0 Modularidad interna: interfaz común entre capas

Cada capa de análisis es un módulo independiente con la misma interfaz: entrada común (*diff* normalizado + metadatos) y salida común (`{ risk_score: 0-100, justification: string }`). Ninguna capa conoce el funcionamiento interno de las demás; el agregador solo las conoce a través de su salida normalizada. Consecuencias prácticas: **activación selectiva** por proyecto (un repositorio sin gestor de paquetes desactiva la capa de dependencias poniendo su peso a 0 en `.watchgate.yml`); **extensión sin reescritura** (la futura capa comportamental

es un módulo nuevo que cumple la misma interfaz); y **sustitución de componentes** (el proveedor de LLM o el motor de reglas pueden cambiarse sin que el resto del sistema lo perciba).

A.0.1 El adaptador de GitHub Action

Con el núcleo sobre git puro, el adaptador queda reducido a «pegamento»: un *workflow* que se dispara en el evento `pull_request`, hace *checkout* con historial completo (`fetch-depth: 0`), obtiene los metadatos de reputación con el `GITHUB_TOKEN` de la propia Action, invoca el mismo binario ya probado en local y publica el comentario y el *check* de CI:

```
# .github/workflows/watchgate.yml
on: { pull_request: { types: [opened, synchronize] } }
steps:
  - uses: actions/checkout@v4
    with: { fetch-depth: 0 }
  - run: watchgate analyze --base ${ github.event.pull_request.base.sha } --head ${ github.sha }
    env: { GITHUB_TOKEN: '${ secrets.GITHUB_TOKEN }', WATCHGATE_LLM_API_KEY: '${ secrets.WATCHGATE_LLM_API_KEY }' }
```

La clave de API del LLM se gestiona como *secret* del repositorio, nunca incrustada en el código. Adoptar WatchGate se reduce a añadir este fichero YAML y configurar el *secret*.

A.1 Flujo de ejecución del sistema

El flujo completo, ilustrado en la Figura 1 (sección 2), es el siguiente:

1. **Trigger de CI:** al abrir/ actualizar un PR (`pull_request` / `merge_request`); produce el *diff* y los metadatos.
2. **Orquestador:** reparte el *diff* y los metadatos a cada capa en paralelo. Es **determinista**: no decide dinámicamente qué capas invocar, sino que lanza siempre las mismas y agrega sus salidas con la fórmula de A.2. Un flujo fijo y auditable es más simple de testear, más barato (una única pasada por capa y PR) y más fácil de justificar que un orquestador agente no determinista. La única agencia del sistema está acotada dentro de la capa semántica (A.3.1).
3. **Capas de análisis, en paralelo:** *3a estática* (Semgrep/YARA sobre el código modificado); *3b dependencias* (paquetes nuevos/modificados contra bases de vulnerabilidades y *typosquatting*); *3c reputación* (antigüedad, historial y consistencia de identidad vía API de la plataforma); *3d semántica* (LLM que razona sobre la intención del cambio, con uso acotado de herramientas, y devuelve JSON con riesgo, categoría y justificación).
4. **Agregador de scoring:** combina las salidas con la fórmula ponderada (A.2) en una puntuación final 0–100 con desglose.
5. **Publicación:** comentario en el PR y *check* de CI con semáforo y desglose explicado.
6. **Feedback humano (opcional):** el revisor marca el veredicto como correcto/falso positivo; ajusta los pesos de futuras ejecuciones en ese repositorio.

A.2 Fórmula de scoring

$Score_{final} = \sum (peso_i \times riesgo_i)$ para $i \in \{estática, dependencias, reputación, semántica\}$

donde *riesgo_i* es la puntuación normalizada (0–100) de cada capa y *peso_i* su ponderación. Los pesos de las capas activas suman 1; si una capa se desactiva (peso 0, ver A.0.0), los restantes se renormalizan para conservar la escala completa 0–100. Pesos por defecto, ajustables por proyecto: **estática 0.25** (fiable pero limitada a patrones conocidos), **dependencias 0.20** (vector complementario), **reputación 0.15** (útil pero falsificable, como demuestra Atomic Arch; por eso recibe el menor peso) y **semántica 0.40** (el mayor peso, porque razona directamente sobre la intención del cambio).

Umbral de semáforo:

- ● **0–30 · Verde:** riesgo bajo; el PR sigue su flujo normal de revisión.
- ● **31–65 · Amarillo:** riesgo medio; se recomienda revisión humana reforzada.
- ● **66–100 · Rojo:** riesgo alto; se bloquea el *merge* si el repositorio así lo configura.

Pesos y umbrales son configurables por repositorio:

```
# .watchgate.yml
weights: { static: 0.25, dependencias: 0.20, reputation: 0.15, semantic: 0.40 }
thresholds: { yellow: 31, red: 66 }
block_on_red: true
```


Así, un proyecto interno con acceso restringido puede relajar el umbral de bloqueo, mientras que un proyecto *open source* muy expuesto puede endurecerlo.

A.3 Formato de salida de la capa semántica (LLM)

```
{
  "risk_score": 72,
  "category": "exfiltracion",
  "justification": "El cambio añade una llamada de red a un dominio externo
                  no declarado en la documentación, activada durante el build.",
  "confidence": "alta"
}
```

Los valores de `category` se limitan a un conjunto cerrado (`exfiltracion`, `backdoor`, `ofuscacion`, `escalada_privilegios`, `ninguna`) para que el agregador los trate de forma consistente sin depender de texto libre.

A.3.1 Uso acotado de herramientas por la capa semántica

Aunque el orquestador es determinista (A.1), dentro de la capa semántica el LLM puede usar herramientas de análisis adicionales antes de emitir su veredicto, cuando el *diff* por sí solo no ofrece contexto suficiente: consultar el registro público de un paquete (npm/PyPI), el historial de *commits* del autor, o el contenido completo de un archivo referenciado fuera del *diff*. Este comportamiento está **acotado** a un máximo de 2 o 3 llamadas por PR (consultas de solo lectura), para que la capa siga siendo predecible en coste y tiempo dentro del pipeline. Use o no herramientas, devuelve siempre la misma salida estructurada de A.3. Este patrón (agencia acotada dentro de una capa, orquestación determinista alrededor) es el mismo que emplean herramientas de revisión de PR como Qodo o CodeRabbit.

A.3.2 RAG local para la capa semántica, bases externas para la de dependencias

Comprobar si una dependencia tiene una vulnerabilidad *conocida* (capa 3b) es una búsqueda exacta (paquete y versión contra un identificador CVE); evaluar si un *diff* se *parece* a un patrón de ataque documentado (capa 3d) es una búsqueda semántica. Un RAG no aporta nada a la primera y es la pieza natural de la segunda. La **capa de dependencias** consulta por defecto fuentes públicas (OSV, GitHub Advisory Database) vía API, con la alternativa configurable de un espejo local sincronizado para entornos *air-gapped*. La **capa semántica** se apoya en un RAG sobre un **corpus local** con resúmenes técnicos de los casos documentados en esta memoria (XZ Utils, prt-scan, Atomic Arch) y entradas de MITRE ATT&CK/CAPEC: antes de invocar al LLM se recuperan por *embeddings* los fragmentos más similares al *diff*, de modo que el modelo pueda razonar «este patrón de ofuscación coincide con la técnica de Atomic Arch» en lugar de evaluar en el vacío. Que el corpus sea local evita fugas de código propietario hacia servicios externos y permite a una empresa sustituirlo o ampliarlo con su propia base de amenazas: WatchGate puede ejecutarse íntegramente sin salir de la red del cliente. Su mantenimiento continuo queda como extensión natural tras el periodo de desarrollo.

A.3.3 Control de coste de tokens en la capa semántica

Al depender de una API de pago por token, el diseño incorpora cuatro mecanismos: **(1) estimación previa** del tamaño del *diff* y su contexto, con truncado o resumen priorizado de los PR que superen un umbral configurable; **(2) caché de resultados** para *diffs* idénticos ya evaluados (p. ej., tras un *rebase* sin cambios); **(3) presupuesto configurable por repositorio** en `.watchgate.yml`, análogo a los pesos de A.2; y **(4) degradación controlada**: si se agota el presupuesto, el agregador calcula el resultado solo con las otras tres capas y lo indica explícitamente en el comentario («capa semántica omitida por límite de presupuesto»), en lugar de fallar en todo-o-nada. Este control acota también el gasto de créditos durante el propio desarrollo y validación (secciones 7 y 8).

A.3.4 Cortocircuitos en los extremos de riesgo (*objetivo ampliado*)

La capa semántica es la más costosa y no siempre es necesaria: el sistema admite, de forma opcional y configurable, dos cortocircuitos. En el **extremo de alto riesgo**, si estática + dependencias + reputación ya superan por sí solas el umbral rojo, el PR se marca en rojo sin invocar al LLM: el resultado es igual o más conservador que el que produciría la capa semántica, así que no reabre ninguna vulnerabilidad (un administrador puede forzar su ejecución igualmente para conservar la justificación en lenguaje natural, a efectos de auditoría). En el **extremo de bajo riesgo**, la aprobación directa exige **dos condiciones, no una**: una puntuación combinada muy por debajo del umbral verde habitual, y la ausencia de patrones que fuerzan siempre la capa semántica, como cambios en

scripts de compilación o CI (PKGBUILD, .github/workflows/, Makefile, Dockerfile), nuevas llamadas de red o nuevas dependencias. Sin esa segunda condición, un ataque tipo Atomic Arch (identidad falsificada, sin tocar dependencias ni disparar reglas estáticas) pasaría sin evaluación semántica, reabriendo justo la vulnerabilidad que motiva el proyecto. Un **muestreo de auditoría** configurable (p. ej., 1 de cada 20 PR que cumplirían el criterio de salto) se envía igualmente a la capa semántica para vigilar si el criterio deja pasar patrones que debería atrapar y recalibrarlo con el tiempo.

A.4 Dashboard de postura de seguridad

Modelo de datos. El agregador ya produce, por ejecución, una puntuación con su desglose; el dashboard solo requiere persistirla, junto con el identificador de PR, el repositorio y la marca de tiempo, en una base de datos relacional simple. El ciclo de *feedback* humano añade una columna de veredicto sobre la misma tabla. No se requiere ningún cambio en las capas ni en el agregador: el dashboard es un consumidor adicional de un dato que el sistema ya genera.

Autenticación (SSO/OAuth). Se delega en un proveedor externo, por dos motivos: reduce la superficie de ataque (que el panel de una herramienta de ciberseguridad gestionara contraseñas débiles sería una contradicción notable) y facilita la adopción en empresas con SSO corporativo. Dos vías no excluyentes: **GitHub/GitLab OAuth** (el usuario entra con su cuenta de la plataforma y el dashboard reutiliza los permisos que ya tiene sobre cada repositorio) y **proveedor OIDC genérico** (Auth0, Keycloak) para identidad corporativa (Okta, Azure AD).

Roles granulares por proyecto. Los permisos se definen por repositorio, reflejando los permisos nativos de GitHub/GitLab:

Permiso	Admin. organización	Mantenedor	Revisor (solo lectura)
Alcance	Todos los repos	Repos asignados	Repos asignados
Ver histórico y desglose	✓	✓	✓
Ajustar pesos/umbrales	✓	✓ (sus repos)	×
Resolver <i>feedback</i> humano	✓	✓	×
Gestionar accesos	✓	×	×

Con OAuth de plataforma, el rol por defecto se infiere del permiso que el usuario ya tiene sobre el repositorio (escritura → mantenedor; lectura → revisor), con ajuste manual solo para casos especiales. **Prioridad interna** si alguna pieza necesitara simplificarse, sin afectar a las funciones núcleo: (1) modelo de datos histórico y vista de solo lectura; (2) GitHub OAuth con roles de mantenedor/revisor; (3) proveedor OIDC genérico y rol de administrador de organización.

Referencias y enlaces técnicos

Casos y campañas citados. Wiz Research, *pvt-scan: AI-Powered GitHub Actions Supply Chain Attack*: <https://www.wiz.io/blog/six-accounts-one-actor-inside-the-pvt-scan-supply-chain-campaign> · StepSecurity, *400+ AUR Packages Hijacked: the “Atomic Arch” Campaign*: <https://www.stepsecurity.io/blog/400-aur-packages-hijacked-atomic-arch-campaign> · SuaInternet, hallazgo de Nicolas Boichat con Gemma E2B: <https://suainternet.com/arch-linux-aur-malware-2026-ataque-sofisticado-1500-pacotes/>

Estado del arte. Datadog Security Labs, *BewAIre: detección de código malicioso en PRs con LLMs*: <https://www.datadoghq.com/blog/engineering/scaling-malicious-code-detection/> · GuardDog (open source): <https://github.com/DataDog/guarddog> · Socket.dev para GitHub: <https://docs.socket.dev/docs/socket-for-github> · Aikido Security, comparativa de detección de malware en dependencias: <https://www.aikido.dev/blog/top-tools-to-detect-malware-in-dependencies>

Recursos técnicos. Semgrep: <https://semgrep.dev> · OSV: <https://osv.dev> · GitHub Advisory Database: <https://github.com/advisories>